

GEOG 247/347 Assignment 7

Zachary Yildiz-Raslan

Part 1

```
#install.packages("sf")
```

```
#install.packages("spdep")
```

```
#install.packages("tmap", dependencies = TRUE)
```

```
library(sf)
library(sp)
library(spdep)
library(tmap)
data(meuse)
```

```
meuse_sf <- st_as_sf(meuse, coords = c("x", "y"), crs = 28992)
coords <- st_coordinates(meuse_sf) # Extracts the coordinate matrix

tm_shape(meuse_sf) +
  tm_dots(col = "zinc",
          style = "quantile",
          palette = "Reds",
          size = 0.3) +
  tm_scalebar(position = c("left", "bottom")) +
  tm_layout(title = "Zinc Concentration in Meuse",
            frame = FALSE)
```

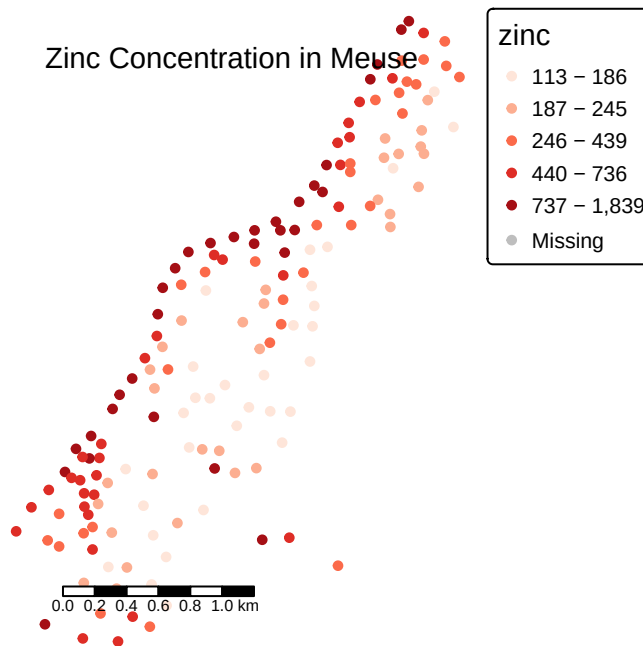
```
-- tmap v3 code detected -----
```

```
[v3->v4] `tm_dots()` : instead of `style = "quantile"`, use fill.scale =
`tm_scale_intervals()``.
i Migrate the argument(s) 'style', 'palette' (rename to 'values') to
  'tm_scale_intervals(<HERE>)'
[v3->v4] `tm_dots()` : use 'fill' for the fill color of polygons/symbols
(instead of 'col'), and 'col' for the outlines (instead of 'border.col').
[v3->v4] `tm_layout()` : use `tm_title()` instead of `tm_layout(title = )`
[cols4all] color palettes: use palettes from the R package cols4all. Run
`cols4all::c4a_gui()` to explore them. The old palette name "Reds" is named
"brewer.reds"
```

Multiple palettes called "reds" found: "brewer.reds", "matplotlib.reds". The first one, "brewer.reds"

[plot mode] fit legend/component: Some legend items or map components do not fit well, and are therefore rescaled.

i Set the tmap option `component.autoscale = FALSE` to disable rescaling.



```
nb_dist <- dnearneigh(coords, d1 = 0, d2 = 500) # define neighbors
listw_dist <- nb2listw(nb_dist, style = "W") # construct W
listw_dist
```

Characteristics of weights list object:

Neighbour list object:

Number of regions: 155
Number of nonzero links: 3202
Percentage nonzero weights: 13.32778
Average number of links: 20.65806

Weights style: W

Weights constants summary:

	n	nn	S0	S1	S2
W	155	24025	155	18.06059	626.0255

```
moran.test(meuse$zinc, listw_dist)
```

Moran I test under randomisation

data: meuse\$zinc
weights: listw_dist

Moran I statistic standard deviate = 9.314, p-value < 2.2e-16

alternative hypothesis: greater

sample estimates:

Moran I statistic	Expectation	Variance
0.2323930989	-0.0064935065	0.0006578318

Task a

The Moran's I value is 0.2323930989 which means this data exhibits a mild level of positive autocorrelation. The p-value is < 2.2e-16 which means this result is definitely significant, given an alpha value of 0.05.

Task b

```
nb_dist2 <- dnearneigh(coords, d1 = 0, d2 = 1000) # define neighbors  
listw_dist2 <- nb2listw(nb_dist2, style = "W") # construct W  
listw_dist2
```

Characteristics of weights list object:

Neighbour list object:

Number of regions: 155

Number of nonzero links: 8518

Percentage nonzero weights: 35.45473
Average number of links: 54.95484

Weights style: W

Weights constants summary:

	n	nn	S0	S1	S2
W 155	24025	155	5.947726	624.1864	

```
moran.test(meuse$zinc, listw_dist2)
```

Moran I test under randomisation

data: meuse\$zinc
weights: listw_dist2

Moran I statistic standard deviate = 1.2115, p-value = 0.1129
alternative hypothesis: greater
sample estimates:

Moran I statistic	Expectation	Variance
0.0088587422	-0.0064935065	0.0001605861

When the distance is changed to 1000 the Moran's I value is 0.0088587422 which is very close to zero (no spatial autocorrelation, random distribution). The p-value you has also changed to 0.1129 which means this result is not statistically significant, given a 0.05 alpha. The difference is due to the change in distance that is used to define nearest neighbors. When the value is set to 500 meters, the Moran test is more likely to pick up on values that are closer to each other and more spatially related. When the distance value is set to 1000 meters, the Moran test is more pick up on values that are far enough away from each other that they are less related and not spatially autocorrelated.

Task c

```
knn <- knearneigh(coords, k = 4)
nb_knn <- knn2nb(knn)
listw_knn <- nb2listw(nb_knn, style = "W")
listw_knn
```

Characteristics of weights list object:
Neighbour list object:

```

Number of regions: 155
Number of nonzero links: 620
Percentage nonzero weights: 2.580645
Average number of links: 4
Non-symmetric neighbours list

```

```

Weights style: W
Weights constants summary:
      n      nn  S0      S1      S2
W 155 24025 155 69.125 639.625

```

```

moran.test(meuse$zinc, listw_knn)

```

Moran I test under randomisation

```

data:  meuse$zinc
weights: listw_knn

```

```

Moran I statistic standard deviate = 9.5334, p-value < 2.2e-16
alternative hypothesis: greater

```

sample estimates:

Moran I statistic	Expectation	Variance
0.493674863	-0.006493506	0.002752549

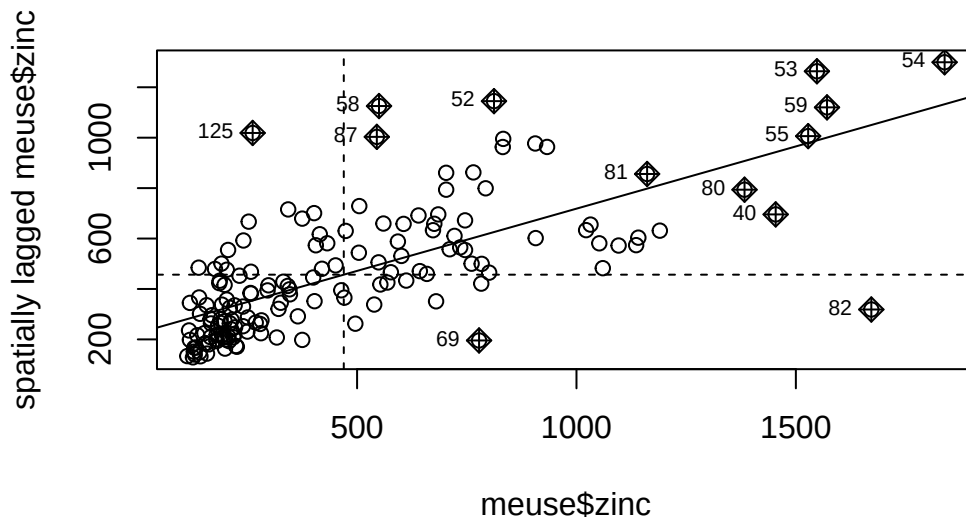
When switching to the K-nearest neighbors approach for the spatial weights matrix, there are significant changes (at least from the 1000 meter distance W). In this KNN Moran's I test, the value is 0.493674863 which is a moderate level of positive spatial autocorrelation (+1 being the highest level). The p-value is $< 2.2e-16$ which means this test is definitely statistically significant, given an alpha value of 0.05. These changes are caused by the KNN spatial weights matrix, defining 4 nearest neighbors. Now, the matrix is evaluating the spatial relationships at a much smaller scale than either 500 or 1000 meter distance weight matrix. Now, instead of evaluating the relationships between all points within 500 or 1000 meters, the Moran's I is now only evaluating the nearest four neighbors.

Task d

```

moran.plot(meuse$zinc, listw_knn)

```



It appears that the LL quadrant is the most populated, followed by HH. This means that the plot is exhibiting obvious positive spatial autocorrelation in the dataset. Since the LL quadrant appears to be the most populated, the spatial pattern of zinc in the KNN weighted matrix is predominantly lower values nearby lower values.

Part 2

Task a

```
library(sf)
library(spdep)
library(tmap)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

nc$SID_rate <- nc$SID79 / nc$BIR79 * 1000

tm_shape(nc) +
  tm_polygons(col = "SID_rate",
              style = "quantile",
              palette = "Reds",
              border.alpha = 0) +
```

```
tm_scale_bar(position = c("left", "bottom")) +
tm_compass(type = "4star", position = c("left", "top")) +
tm_layout(main.title = "SIDS Rate in North Carolina Counties, 1979",
           main.title.size = 0.95,
           frame = FALSE)
```

-- tmap v3 code detected -----

[v3->v4] `tm\_polygons()`: instead of `style = "quantile"`, use `fill.scale =`
`tm_scale_intervals()`.

i Migrate the argument(s) 'style', 'palette' (rename to 'values') to
`tm_scale_intervals(<HERE>)`

[v3->v4] `tm\_polygons()`: use `col_alpha` instead of `border.alpha`.

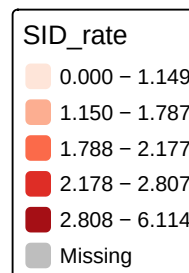
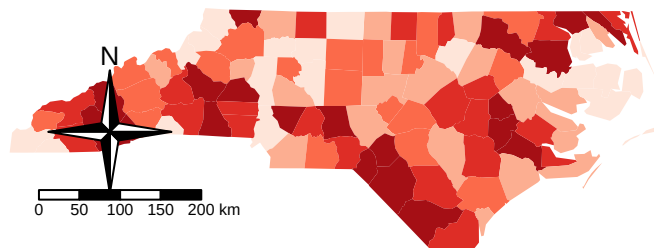
! `tm_scale_bar()` is deprecated. Please use `tm_scalebar()` instead.

[v3->v4] `tm_layout()`: use `tm_title()` instead of `tm_layout(main.title = )`

[cols4all] color palettes: use palettes from the R package `cols4all`. Run
`cols4all::c4a_gui()` to explore them. The old palette name "Reds" is named
`"brewer.reds"`

Multiple palettes called "reds" found: "brewer.reds", "matplotlib.reds". The first one, "bre

SIDS Rate in North Carolina Counties, 1979



```
nc1 <- st_transform(nc, 32119)
coords1 <- st_coordinates(st_centroid(nc1))
```

Warning: st_centroid assumes attributes are constant over geometries

```
nb_dist3 <- dnearneigh(coords1, d1 = 0, d2 = 50000) # define neighbors
listw <- nb2listw(nb_dist3, style = "W") # construct W
listw
```

Characteristics of weights list object:

Neighbour list object:

Number of regions: 100

Number of nonzero links: 432

Percentage nonzero weights: 4.32

Average number of links: 4.32

Weights style: W

Weights constants summary:

	n	nn	S0	S1	S2
W	100	10000	100	50.37864	405.787

```
moran.test(nc$SID_rate, listw)
```

Moran I test under randomisation

data: nc\$SID_rate

weights: listw

Moran I statistic standard deviate = 2.7179, p-value = 0.003285

alternative hypothesis: greater

sample estimates:

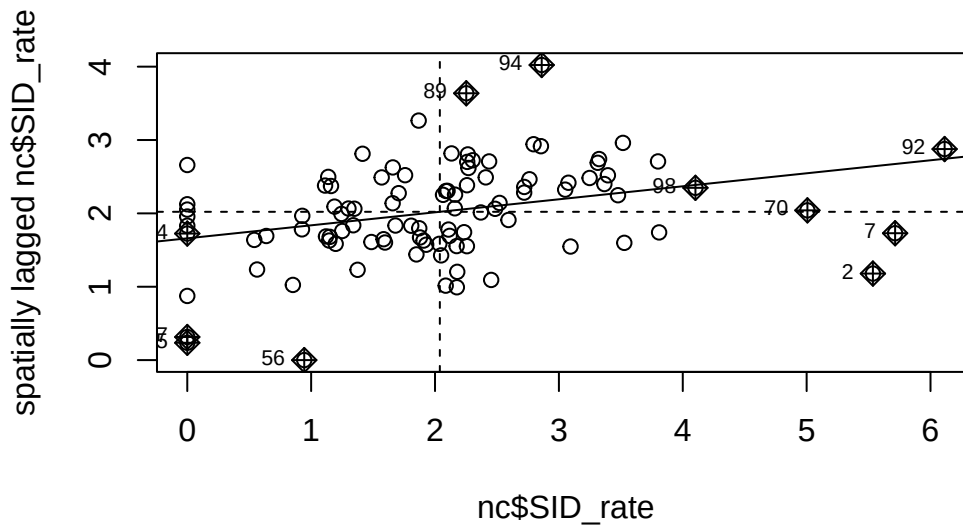
Moran I statistic	Expectation	Variance
0.177298309	-0.010101010	0.004754227

Any of the three methods to define neighbors discussed in class can be used for this dataset. Because this is a polygon dataset, continuity (queen or rook), distance-based (using polygon centroids), and K nearest neighbors can all be used on this dataset. For this task, I chose to use the distance based spatial relationship to construct the spatial weights matrix W. I transformed the data to the NC State Plane (Meters) PCS and set the distance to 50km. With

these parameters, the Moran's I statistic is 0.177298309 with a p-value of 0.003285. The sign of the Moran's I statistic is positive, indicating positive spatial autocorrelation. This means that similar values are near each other.

Task b

```
moran.plot(nc$SID_rate, listw)
```



The Moran scatterplot for SID_rate using the listw weights matrix appears to show HH as the most populated quadrant, though it is difficult to be sure just by looking at this plot - both HH and LL look similarly populated. If the HH quadrant is the more populated one (which I think it is), this would mean that counties, within the defined neighborhood, with high rates of SIDS are near other counties with high rates of SIDS.

Task c

```
local_sid <- localmoran(nc$SID_rate, listw)
```

```
nc$local_I <- local_sid[, 1] # Local Moran's I values
nc$local_Z <- local_sid[, 4] # Z-scores
nc$local_p <- local_sid[, 5] # p-values
```

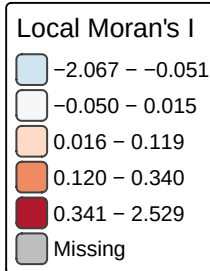
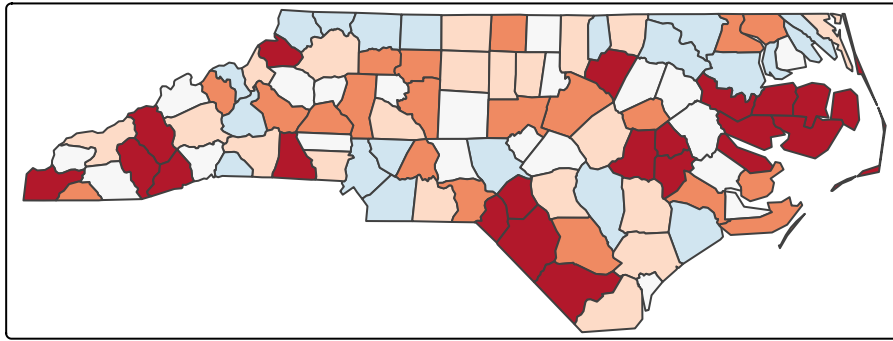
```
# Moran's I values
localI_map <- tm_shape(nc) +
  tm_polygons(col = "local_I",
              title = "Local Moran's I",
              style = "quantile",
              palette = "-RdBu") +
  tm_layout(legend.outside = TRUE)
```

-- tmap v3 code detected -----

```
[v3->v4] `tm_polygons()`: instead of `style = "quantile"`, use fill.scale =
`tm_scale_intervals()`.
i Migrate the argument(s) 'style', 'palette' (rename to 'values') to
  'tm_scale_intervals(<HERE>)'
[v3->v4] `tm_polygons()`: migrate the argument(s) related to the legend of the
visual variable `fill` namely 'title' to 'fill.legend = tm_legend(<HERE>)'
```

```
localI_map
```

Multiple palettes called "rd_bu" found: "brewer.rd_bu", "matplotlib.rd_bu". The first one, "I
[scale] tm_polygons(): the data variable assigned to 'fill' contains positive and negative v
[cols4all] color palettes: use palettes from the R package cols4all. Run
`cols4all::c4a\_gui()` to explore them. The old palette name "-RdBu" is named
"rd_bu" (in long format "brewer.rd_bu")



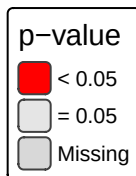
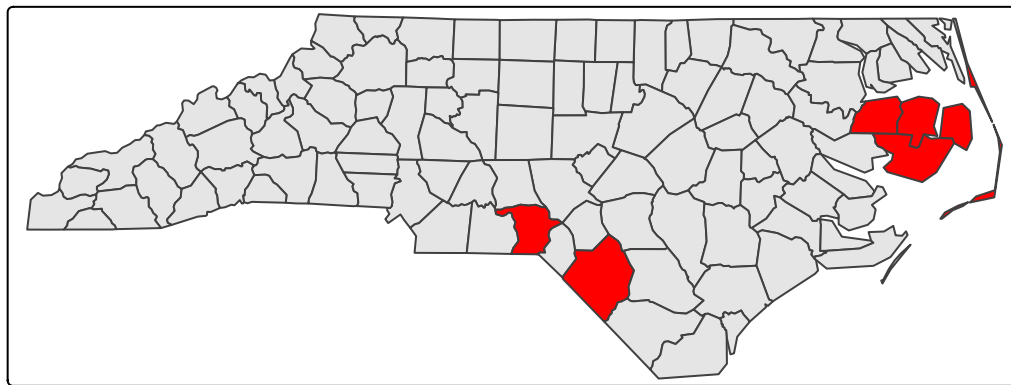
```
# p-values
localp_map <- tm_shape(nc) +
  tm_polygons(col = "local_p",
              title = "p-value",
              breaks = c(-Inf, 0.05, Inf),
              palette = c("red", "grey90")) +
  tm_layout(legend.outside = TRUE)
```

-- tmap v3 code detected -----

[v3->v4] `tm\_tm\_polygons()`: migrate the argument(s) related to the scale of the visual variable `fill` namely 'breaks', 'palette' (rename to 'values') to `fill.scale = tm_scale(<HERE>)`.

[v3->v4] `tm\_polygons()`: migrate the argument(s) related to the legend of the visual variable `fill` namely 'title' to `fill.legend = tm_legend(<HERE>)`

localp_map



```
lag_sid <- lag.listw(listw, nc$SID_rate)

nc$cluster <- ifelse(nc$local_p > 0.05, "Not significant",
  ifelse(nc$SID_rate > mean(nc$SID_rate) & lag_sid > mean(nc$SID_rate), "HH",
  ifelse(nc$SID_rate < mean(nc$SID_rate) & lag_sid < mean(nc$SID_rate), "LL",
  ifelse(nc$SID_rate > mean(nc$SID_rate) & lag_sid < mean(nc$SID_rate), "HL", "LH")),
  "Not significant")

nc$cluster <- factor(nc$cluster,
  levels = c("HH", "LL", "HL", "LH", "Not significant"))
```

```
cluster_map <- tm_shape(nc) +
  tm_polygons(col = "cluster",
    title = "LISA Cluster",
    palette = c("red", "lightblue", "orange", "blue", "grey90")) +
  tm_layout(legend.outside = TRUE)
```

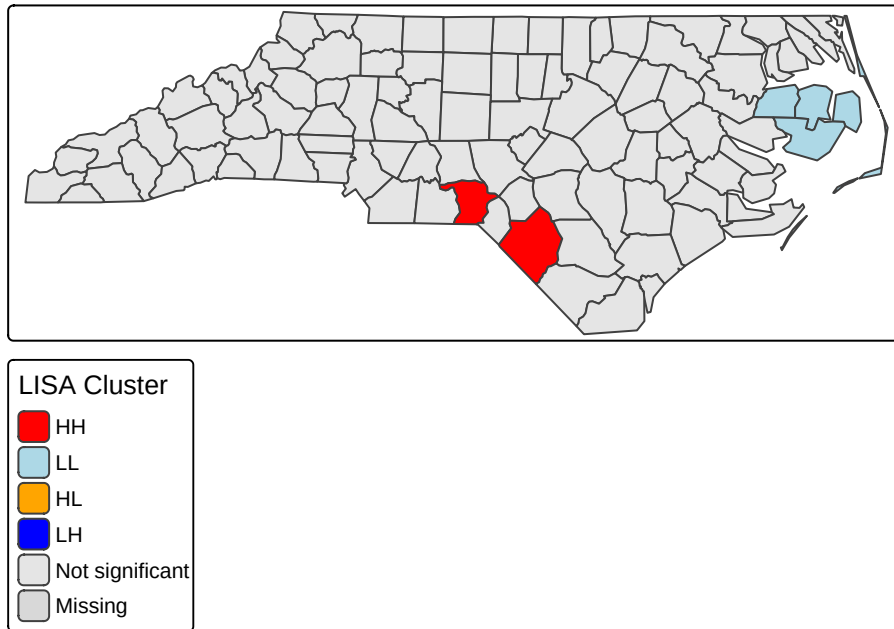
-- tmap v3 code detected -----

[v3->v4] `tm\_tm\_polygons()`: migrate the argument(s) related to the scale of

```
the visual variable `fill` namely 'palette' (rename to 'values') to fill.scale  
= tm_scale(<HERE>).
```

```
[v3->v4] `tm_polygons()`: migrate the argument(s) related to the legend of the  
visual variable `fill` namely 'title' to 'fill.legend = tm_legend(<HERE>)'
```

```
cluster_map
```



When visually comparing the Local Moran's I map with the P-Value map, I can see that the those counties that are statistically significant are also HH and LL counties. Although it is not the case with my current weight matrix and this dataset, it is possible to have a county with large positive or negative Local Moran's I value while not being statistically significant at the 0.05 alpha level (however, this is not the case for the LISA Cluster map). In the context of Local Moran's I, the p-value indicates the likelihood of observing the Local Moran's I value by chance (under the null hypothesis of no spatial autocorrelation).

Task d

No, a positive value for the global Moran's I for SID_rate does not mean every county belongs to a cluster. In my case, the Moran's I value is positive, which means that this dataset does have positive spatial autocorrelation. LISA Clusters are defined by the Moran scatterplot quadrant and the significance test for Local Moran's I. Most counties in this dataset do not have statistically significant p-values, even though they may have positive spatial autocorrelation. Only counties that are statistically significant will belong to a cluster (as seen in the LISA Cluster

map). The positive global Moran's I for SID_rate indicates that there is spatial autocorrelation in this dataset, not that every county is significantly positively spatially autocorrelated.

Extra Credit Portion

```
moran.test(meuse$zinc, listw_dist)
```

Moran I test under randomisation

```
data: meuse$zinc
weights: listw_dist
```

```
Moran I statistic standard deviate = 9.314, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:
Moran I statistic      Expectation      Variance
      0.2323930989      -0.0064935065      0.0006578318
```

```
local_sid2 <- localmoran(meuse$zinc, listw_dist)
```

```
meuse$local_I <- local_sid2[, 1] # Local Moran's I values
meuse$local_Z <- local_sid2[, 4] # Z-scores
meuse$local_p <- local_sid2[, 5] # p-values
```

```
lag_sid1 <- lag.listw(listw_dist, meuse$zinc)
```

```
meuse$cluster <- ifelse(meuse$local_p > 0.05, "Not significant",
  ifelse(meuse$zinc > mean(meuse$zinc) & lag_sid1 > mean(meuse$zinc), "HH",
  ifelse(meuse$zinc < mean(meuse$zinc) & lag_sid1 < mean(meuse$zinc), "LL",
  ifelse(meuse$zinc > mean(meuse$zinc) & lag_sid1 < mean(meuse$zinc), "HL", "LH"))
```

```
meuse$cluster <- factor(meuse$cluster,
  levels = c("HH", "LL", "HL", "LH", "Not significant"))
```

```
meuse_sf$cluster <- meuse$cluster
cluster_map <- tm_shape(meuse_sf) +
  tm_dots(col = "cluster",
  title = "LISA Cluster",
```

```

        palette = c("red", "blue", "orange", "lightblue", "grey90"),
        size = 0.4) +
tm_layout(legend.outside = TRUE)+
tm_scalebar(position = c("right", "bottom")) +
tm_layout(title = "LISA Cluster of Zinc in Meuse",
          frame = TRUE)

```

-- tmap v3 code detected -----

[v3->v4] `tm\_tm\_dots()`: migrate the argument(s) related to the scale of the visual variable `fill` namely 'palette' (rename to 'values') to fill.scale = tm_scale(<HERE>).

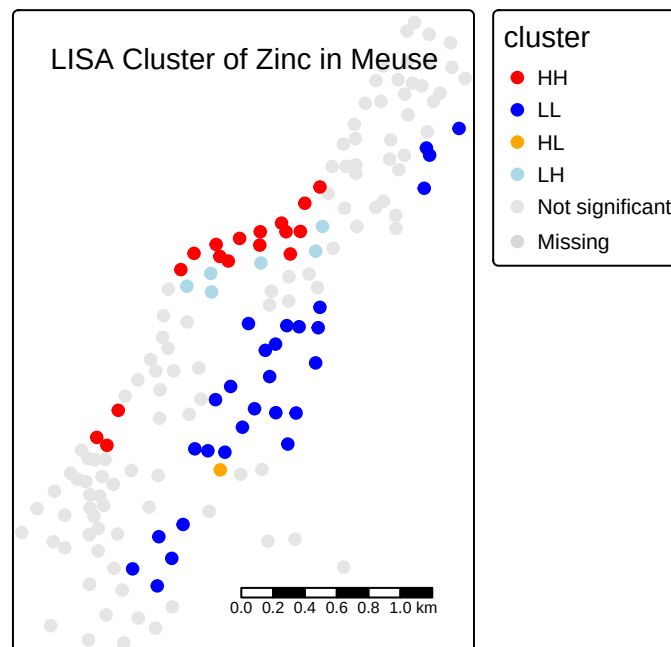
[tm_dots()] Argument `title` unknown.

[v3->v4] `tm\_layout()`: use `tm\_title()` instead of `tm\_layout(title = )`

cluster_map

[plot mode] fit legend/component: Some legend items or map components do not fit well, and are therefore rescaled.

i Set the tmap option `component.autoscale = FALSE` to disable rescaling.



From the LISA Cluster map, I see the most predominant LISA cluster value is LL which spans along the right side of the dataset from south to north. This means most significant clustering is of low zinc concentration areas near other low zinc concentration areas. There are also two clusters of HH observations in the top the middle of the dataset, the larger one being further north of the smaller area. These areas are high zinc concentration areas near other high zinc concentration areas. It is clear from the LISA cluster map, as it is in the Moran's I table at the beginning of this section, that there is a large amount of positive spatial autocorrelation in this dataset. There is a smaller amount of negative spatial autocorrelation (outlier observations) as well, with a string of LH points visible just under the HH section toward the center of the map. There is also one point that has a HL value as well, but the negative spatial autocorrelated values are much less prevalent in this dataset. These fewer negatively spatially autocorrelated points indicate that they are near values that are dissimilar to them. Throughout the center of the dataset, from top to bottom, there are mostly statistically insignificant points.